

计算方法B

9.23应交上机作业

宋思逸

2400010816

2025 年 9 月 19 日

0

由于本人暂时对MATLAB这种语言并不熟悉，因此最初的几次作业可能都会用C语言或C++来描述。此外，我会使用freopen函数来直接对输入输出进行操作。压缩包中包含了代码和保存了输出结果的txt文档

1

1.1 不选主元Gauss消元法

我们需要无脑实现最基本的Gauss消元法

我们通过函数void gaussElimination来实现这一功能

Gauss变换需要使得主对角线下方的所有元素均变为0。这里为了使过程更简洁我们使用了高等代数中的增广矩阵，直接通过初等行变换（即Gauss消去）进行操作。

当主对角线上元素不为0时，我们用factor存储系数因子（可将第k,k元素下面的元素全部变成0），对下面的行进行操作，随后再反过来操作一遍，即可得到方程组的解

这里我们发现，尽管在操作原理上并没有问题，但是求解的结果却出现了比较大的偏差。题目中要求的84元方程组精确解为 $(1, 1, \dots, 1)$ ，但是最后解出来 x_{84} 竟然达到了惊人的536838145，这个误差是我们不能接受的。通过保留20位小数的操作，我们发现其实 x_2 的求解就已经有了误差，这个

误差在每一步中越积越大，最终造成了巨大的影响

这就让我们去寻找一种更优的解法以减小误差，即题目中所提及的列主元Gauss消去法

1.2 列主元Gauss消去法

我们首先使用函数swapRows进行交换行的操作，即将第row1行和第row2行进行交换

接下来我们使用columnPivotGauss函数进行一个列主元Gauss消去的处理

首先我们用打擂台法找出最大值，这里我们需要用maxRow保存某一列上最大值所在的行，maxVal保存这个最大值。当我们发现maxRow不是k的时候，就需要将maxRow与k进行交换，从而可以实现选取列主元的操作

接下来就按照一般的Gauss消去法来进行操作了，即将主元下方的全消成0，以及自下而上带入求解的过程

我们发现在此操作模式下，原先极大的误差得到了控制，通过保留20位小数可以看到误差数量级在 10^{-5} （ $n=84$ ）时，这个已经比刚才的要好得多

1.3 小结

Gauss消元法的原理虽然简单且清晰，但是在实际操作的过程中，由于计算机中的小数是以浮点数的形式保存，与真实值存在误差，因此我们要想办法（比如这题中的选主元）来尽可能减小这种误差

2 平方根法和改进的平方根法

这里换了一个编译器编写。

首先我们需要计算这个系数矩阵的Cholesky分解。这没什么好说的，按照课堂上的内容直接操作一遍就完事了。然后我们分别计算 $Ly = b$ 和 $L^T x = y$ 的解，即代码中所展现的forward_substitution和backward_substitution。总之就是比较系数和暴力计算吧。观察到保留20位小数的情况下计算结果居然是收敛的，也就是最后完全不动了，误差在 10^{-10} 以内

优化的Cholesky分解也没什么好说的，按照上课所讲和书本内容直接操作就完了。最后结果也相当的稳定，在保留20位小数的情况下最后完全

收敛到精确解了，误差在 10^{-15} 以内，还是相当精确的

上述针对的是第一个例子，我们还需要来看一下第二个例子，也就是传说中的Hilbert矩阵。它的定义是

$$a_{ij} = \frac{1}{i+j-1}$$

根据题目中给到的 $b_{ij} = \sum_{j=1}^n \frac{1}{i+j-1}$ 很容易看出精确解为 $x_i = 1, \forall i = 1, 2, \dots, 40$

然而我们发现最后的解简直稀奇古怪，除了 x_1, x_2, x_3, x_4 勉强还能看出来是在1附近摆动之外，其他的解简直就是到处乱蹦，甚至让我一度怀疑自己的代码是不是写错了。最后在AI工具的帮助下发现这个矩阵实在是比较的病态，因此最后跑出来错的离谱的结果也就不难理解了。

3 小结

总结一下这次写代码的收获吧。其实我一开始不怎么理解为什么这个课要把上课讲过的代码再写一遍自己运行一遍。但事实上正如张老师在课堂上所提到的，之前大一学习的例如数分高代等课程的理论是很好的，但是在实际操作的过程中会有比较大的问题，例如一些定理中的“存在”我要怎么去找到或者构造这个存在的东西，真正在计算的时候无法避免的误差如何尽可能减小，都是我们学习这门课程的一个意义和目的所在

压缩包中附上代码和运行结果，这里就不在报告当中赘述了